Projekt: Software Engineering

(DLMCSPSE01_D)

IU Internationale Hochschule

Studiengang: Informatik

Projektdokumentation

Jessica Krautheuser

Matrikelnummer: IU14129800

Tutor: Prof. Dr. Dirk Simon

Abgabedatum: 30.10.2026

Inhalt

Tabellenverzeichnis III

Abkürzungsverzeichnis IV

1. Projektidee 1

2. Projektmotivation 1

3. Projektmethodik 1

4. Projektübersicht 3

4.1 Projektziel 3

4.2 Spielprinzip 3

4.3 Zielgruppe 4

4.4 Kundennutzen 4

4.5 Projektumfang 4

5. Risikomanagement 6

6. Zeitplanung 8

Literaturverzeichnis IV

Tabellenverzeichnis

Tabelle 1: Risikomatrix 6
Tabelle 2: Arbeitspakete 8
Tabelle 3: Zeitplan 9
Tabelle 4: Meilensteine 9

Abkürzungsverzeichnis

A	Auswirkung
AP	Arbeitspaket
E	Eintrittswahrscheinlichkeit
GUI	Graphic User Interface
M	Meilenstein

1. Projektidee

Im Rahmen des Projekts wird ein Spiel mit dem Titel Mystic Maze entwickelt. Das grundlegende Spielprinzip orientiert sich an dem Gesellschaftsspiel „Das verrückte Labyrinth“ des Ravensburger Verlags (Ravensburger AG, o. J.). Dabei wird ein aus einzelnen Kacheln bestehendes Spielfeld während des Spielverlaufs dynamisch verändert. Die Spieler müssen das Labyrinth gezielt verschieben, um Wege zu bestimmten Zielen zu öffnen und gleichzeitig die Möglichkeiten der Mitspielenden einzuschränken.

Im Gegensatz zum grundlegenden Spielprinzip von „Das verrückte Labyrinth“ wird Mystic Maze insbesondere durch Spezialkacheln sowie individuelle Charakterfähigkeiten erweitert. Dadurch entstehen zusätzliche taktische Entscheidungsmöglichkeiten und ein eigenständiges Spielkonzept.

Die Anwendung wird in Python entwickelt und als Webanwendung bereitgestellt.

Der Quellcode und die zugehörigen Dateien des Projekts werden in einem öffentlichen GitHub-Repository bereitgestellt. Dieses ist unter https://github.com/mysticmazejk/Mystic_Maze abrufbar.

2. Projektmotivation

Die Motivation für Mystic Maze ergibt sich aus dem persönlichen Interesse an strategischen Gesellschaftsspielen und dem Ziel, ein bereits bestehendes Spielprinzip durch eigene Ideen weiterzuentwickeln und dabei Kenntnisse des Software Engineerings praktisch anzuwenden. Die Umsetzung als browserbasierte Mehrspieler-Anwendung bietet dabei die Möglichkeit, unterschiedliche Herausforderungen wie komplexe Spiellogik, Client-Server-Kommunikation, Synchronisation, Testbarkeit und eine klare Trennung von Logik und Benutzeroberfläche praktisch zu bearbeiten. Gleichzeitig können zentrale Methoden des Requirements Engineering, der UML-Modellierung, der Softwarearchitektur und der Qualitätssicherung in einem zusammenhängenden Projekt angewendet werden.

Aus der Projektidee und der Projektmotivation ergibt sich folgende übergeordnete Projektfrage:

Wie kann das beschriebene Spiel Mystic Maze mithilfe geeigneter Methoden des Software Engineerings als wartbare, testbare und mehrspielerfähige Webanwendung konzipiert und umgesetzt werden?

3. Projektmethodik

Für die Entwicklung von Mystic Maze werden sowohl klassische als auch agile Methoden des Software Engineerings verwendet. Klassische Vorgehensweisen zeichnen sich durch eine strukturierte Planung, definierte Entwicklungsaktivitäten und eine vergleichsweise umfangreiche Dokumentation aus. Agile Vorgehensweisen setzen dagegen stärker auf iterative Entwicklung, regelmäßiges

Feedback und die Anpassung an neue Erkenntnisse (Mishra & Alzoubi, 2023, S. 1504-1506). Die Kombination beider Ansätze wird als hybrides Vorgehen bezeichnet. Dabei können strukturierte Vorgehensmodelle als Rahmen dienen und durch agile Praktiken ergänzt werden (Kuhrmann et al., 2017, S. 30-31) . Für Mystic Maze ermöglicht dieses Vorgehen, die Planungssicherheit der vorgegebenen Projektphasen mit der notwendigen Flexibilität während der technischen Entwicklung zu verbinden.

Das Projekt gliedert sich in die Konzeptionsphase, die Erarbeitungs- und Reflexionsphase sowie die Finalisierungsphase. In der Konzeptionsphase werden Projektplanung, Anforderungsermittlung und Spezifikation durchgeführt. Die Projektplanung definiert unter anderem Projektumfang, Arbeitspakete, Meilensteine, Risiken und Zeitplanung. Bei der Anforderungsanalyse werden anschließend die Anforderungen der relevanten Stakeholder ermittelt, analysiert und dokumentiert (Nuseibeh & Easterbrook, 2000, S. 37). Die Spezifikation konkretisiert diese Anforderungen durch Geschäftsobjekte, Geschäftsprozesse, Geschäftsregeln und Schnittstellen. Diese klassische Grundlage wird bereits in der Konzeptionsphase durch agile Elemente ergänzt. Dazu gehört insbesondere die Beschreibung funktionaler Anforderungen durch User Stories. Diese stellen Anforderungen aus Sicht einer bestimmten Rolle kompakt dar und werden durch Akzeptanzkriterien konkretisiert (Lucassen et al., 2016, S. 383–384). Bei Mystic Maze werden daher alle User Stories um überprüfbare Akzeptanzkriterien ergänzt.

In der Erarbeitungs- und Reflexionsphase werden zunächst die Softwarearchitektur ausgearbeitet und anschließend einzelne Funktionen schrittweise implementiert, getestet und bei Bedarf angepasst. Erkenntnisse aus Implementierung und Tests können dadurch zu einer Verfeinerung bestehender Anforderungen, Spezifikationen oder Architekturentscheidungen führen. Iterationen sind ein wesentlicher Bestandteil der agilen Softwareentwicklung und können zur Verbesserung der Softwarequalität beitragen, verursachen bei unkontrolliertem Einsatz jedoch auch zusätzlichen Zeit- und Arbeitsaufwand (Mumtaz et al., 2022, S. 65355–65356).

In der Finalisierungsphase wird überprüft, ob die entwickelte Anwendung die zuvor definierten Anforderungen und Akzeptanzkriterien erfüllt. Hierzu werden die durchgeführten Tests ausgewertet und die relevanten Teststufen betrachtet. Unit-Tests prüfen einzelne Komponenten bzw. Funktionen isoliert, während Integrationstests das Zusammenspiel mehrerer Komponenten überprüfen. Systemtests betrachten das Verhalten der vollständigen Anwendung als Gesamtsystem (Di Lucca & Fasolino, 2006, S.1176) . Die konkrete Durchführung und die Ergebnisse dieser Teststufen werden dokumentiert, um nachvollziehbar darzustellen, inwieweit die festgelegten Anforderungen erfüllt wurden. Anschließend wird das Vorgehen, aufgetretene Abweichungen, Limitationen und die Zielerreichung kritisch reflektiert. Die dabei gewonnenen Erkenntnisse werden als Lessons Learned festgehalten, um Erfahrungen aus dem Projekt für zukünftige Entwicklungsprojekte nutzbar zu machen (Abdellatif et al., 2019, S.44).

4. Projektübersicht

4.1 Projektziel

Ziel des Projekts ist die Konzeption und Entwicklung einer vollständig spielbaren Webanwendung, die das beschriebene Labyrinthspiel bereitstellt. Im Mittelpunkt stehen folgende Ziele:

- Entwicklung eines dynamisch veränderbaren Labyrinths auf einem rasterbasierten Spielfeld
- Umsetzung eines vollständigen, regelbasierten Spielablaufs
- Unterstützung von mindestens zwei Spieler an verschiedenen Endgeräten
- Kommunikation und Synchronisation der Spielzustände zwischen mindestens zwei Endgeräten innerhalb desselben lokalen Netzwerks beziehungsweise WLANs
- Integration von normalen und speziellen Labyrinthkacheln
- Integration verschiedener Zielkarten
- Verwendung unterschiedlicher Spielcharaktere mit individuellen Fähigkeiten
- grafische und intuitive Bedienung über Web-Graphic User Interface (GUI)
- saubere Trennung von Spielregeln, Spiellogik und GUI
- Entwicklung einer Anwendung, die automatisiert und manuell getestet werden kann

4.2 Spielprinzip

Zu Beginn einer Partie wird ein Labyrinth aus quadratischen Kacheln erzeugt. Die Kacheln besitzen unterschiedliche Wegverläufe:

- gerade Wege
- Kurven
- T-Kreuzungen
- Kreuzungen

Zusätzlich existieren besondere Kacheln mit zusätzlichen Spieleffekten.

Zu Beginn eines Spielzugs erhält der aktive Spieler eine freie Labyrinthkachel. Diese kann gedreht und anschließend an einer zulässigen Position in das Spielfeld eingeschoben werden. Dadurch verschiebt sich eine vollständige Reihe oder Spalte des Labyrinths und die gegenüberliegende Kachel wird aus dem Spielfeld herausgeschoben.

Anschließend darf sich die Spielfigur des aktiven Spielers über alle miteinander verbundenen Wege bewegen. Ziel ist es, bestimmte auf Zielkarten Artefakte zu erreichen.

Sobald das aktuelle Ziel erreicht wurde, wird die nächste Zielkarte aktiviert. Eine Partie endet, sobald ein Spieler alle ihm zugewiesenen Ziele erreicht hat.

Mystic Maze erweitert dieses grundlegende Spielprinzip insbesondere durch Spezialkacheln und individuelle Charakterfähigkeiten. Dadurch entstehen zusätzliche taktische Entscheidungsmöglichkeiten:

- Spezialkacheln

    - Portal: ermöglicht den Wechsel zu einem zweiten Portal
    - Tor: blockiert einen Weg, bis ein entsprechendes Schlüssel-Artefakt gefunden wurde
- Charakterfähigkeiten
    - Magier: darf einmal je gesuchtem Artefakt eine Kachel auf dem Spielfeld drehen
    - Wächter: darf Tore durchschreiten, ohne das benötigte Schlüssel-Artefakt zu besitzen

4.3 Zielgruppe

Die primäre Zielgruppe besteht aus Personen, die strategische Brett-, Puzzle- und Casual-Games bevorzugen. Angesprochen werden insbesondere:

- Jugendliche und Erwachsene
- Gelegenheitsspieler
- Fans taktischer Gesellschaftsspiele
- Personen, die lokale Mehrspieler-Spiele spielen möchten

Für das Spiel sind keine besonderen technischen Kenntnisse erforderlich.

4.4 Kundennutzen

Der Customer Value von Mystic Maze entsteht durch die Kombination aus leicht verständlicher Grundmechanik und taktischen Entscheidungsmöglichkeiten. Das Spiel kann im Webbrowser gestartet werden und benötigt kein physisches Spielmaterial. Die Software automatisiert dabei einen Teil der Spielorganisation. Ein weiterer Mehrwert entsteht dadurch, dass das Spiel von verschiedenen Personen am eigenen Endgerät gespielt werden kann. Dadurch entfällt die Notwendigkeit, sich einen Bildschirm oder Eingabegeräte zu teilen. Gleichzeitig ist für eine lokale Partie keine Bereitstellung eines öffentlich erreichbaren Internetservers erforderlich.

- automatische Prüfung der Einhaltung der Spielregeln
- automatische Ermittlung erreichbarer Felder
- automatisches Verschieben der Spielkacheln
- automatische Verwaltung von Zielkarten
- automatische Prüfung von Siegbedingungen
- keine Vorbereitung oder Sortierung von Spielmaterial erforderlich
- gemeinsames Spielen über mehrere Endgeräte

4.5 Projektumfang

Für eine kontrollierbare Umsetzung wird zwischen Muss-Funktionen und möglichen Erweiterungen unterschieden.

Geplanter Kernumfang:

- webbasierte GUI
- Spielbrett mit 7 × 7 Kacheln
- unterschiedliche Labyrinthkacheltypen
- Drehung der freien Kachel

- Einschieben von Reihen und Spalten
- automatische Aktualisierung des Spielfelds
- Spielfiguren
- Wegfindung und Prüfung erreichbarer Felder
- Einsammeln von Artefakten
- Zielkarten
- Artefakt-Kacheln
- Portal-Kacheln
- Tor-Kacheln
- zwei unterschiedliche Spielcharaktere mit Sonderfähigkeiten
- Prüfung der Siegbedingung
- Start einer neuen Partie
- zwei Spieler an verschiedenen Endgeräten
- Betrieb der Endgeräte innerhalb desselben lokalen Netzwerks beziehungsweise WLANs
- Erstellen beziehungsweise Hosten einer lokalen Partie
- Beitritt zu einer bestehenden lokalen Partie
- webbasierte Client-Server-Kommunikation
- zentrale Verwaltung des Spielzustands durch den Server
- Synchronisation von Spielaktionen und Spielzuständen zwischen den Endgeräten
- Behandlung eines Verbindungsabbruchs durch eine verständliche Fehlermeldung

Mögliche Erweiterungen:

- Speichern des Spielstandes
- Bis 4 Spieler an verschiedenen Endgeräten
- Computergegner
- weitere Charaktere mit Sonderfertigkeiten
- weitere Spezialkacheln
- Zufallsereignisse
- verschiedene Schwierigkeitsgrade
- Statistiken
- Highscore
- Soundeffekte und Hintergrundmusik

Diese Erweiterungen sind nachrangig. Im Folgenden wird nur die Umsetzung des Kernumfangs betrachtet.

5. Risikomanagement

Softwareprojekte unterliegen technischen, organisatorischen und zeitlichen Risiken. Für Mystic Maze werden die wichtigsten Risiken bereits während der Konzeptionsphase identifiziert und bewertet.

Bewertungsmethode

Für jedes Risiko werden zwei Faktoren auf einer Skala von 1 bis 5 bewertet:

- Eintrittswahrscheinlichkeit (E): 1 = sehr gering, 5 = sehr hoch
- Auswirkung (A): 1 = sehr gering, 5 = sehr schwerwiegend

Aus beiden Werten (E x A) ergibt sich das Risiko

Tabelle 1: Risikomatrix

ID	Risiko	E	A	Risiko	Gegenmaßnahme
R01	Projektumfang wird zu groß	4	5	20	MVP früh festlegen, Erweiterungen klar vom Kernumfang trennen
R02	Spielfeldlogik ist komplexer als erwartet	3	5	15	Spielfeld- und Verschiebelogik früh entwickeln und testen
R03	Fehler bei der Wegfindung	3	4	12	Algorithmus unabhängig von der GUI implementieren und Unit-Tests erstellen
R04	Charakterfähigkeiten führen zu Regelkonflikten	3	3	9	Fähigkeiten begrenzen und Geschäftsregeln eindeutig dokumentieren
R05	GUI benötigt mehr Entwicklungszeit als geplant	3	4	12	zunächst funktionale Minimal-GUI entwickeln, grafische Details später
R06	Projektverzögerung durch unerwartete Fehler	3	4	12	Zeitpuffer vor der Finalisierung einplanen
R07	Unzureichende Testabdeckung	3	4	12	Tests parallel zur Entwicklung statt ausschließlich am Projektende; Einsatz von Unittests während der Entwicklung

Quelle: Eigene Darstellung

Als größtes Risiko wird derzeit ein zu großer Projektumfang mit Risiko 20 bewertet. Die Vielzahl möglicher Funktionen kann dazu führen, dass zu viel Entwicklungszeit für optionale Funktionen aufgewendet wird. Daher wird von Beginn an ein klar definierter Minimalumfang festgelegt.

Ein weiteres wesentliches Risiko liegt in der Spielfeld- und Wegfindungslogik. Da sich das Labyrinth nach jedem Zug verändert, muss zuverlässig bestimmt werden können, welche Kacheln miteinander verbunden sind und welche Felder von einer Spielfigur erreicht werden können. Diese Funktion gehört deshalb zu den ersten zu implementierenden und zu testenden Komponenten.

6. Zeitplanung

Die Entwicklung wird in mehrere Arbeitspakete (AP) gegliedert. Dadurch können Projektfortschritt und mögliche Verzögerungen leichter überwacht werden.

Tabelle 2: Arbeitspakete

AP	**Arbeitspaket**	**Inhalt**
AP01	Projektkonzeption	Spielidee, Zielgruppe, Projektumfang
AP02	Projektdokumentation	Projektübersicht, Risiken, Zeitplanung
AP03	Anforderungsanalyse	Stakeholder, User Stories, Qualitätsanforderungen
AP04	Spezifikation	Datenmodell, Prozesse, Regeln und Schnittstellen
AP05	Architektur	Technologieauswahl und Softwarearchitektur
AP06	Implementierungs-setup	Entwicklungsumgebung einrichten
AP07	Grundlegende Spiello-gik	Spielfeld, Kacheln, Spieler, Spielzustand
AP08	Labyrinthmechanik	Drehen und Verschieben von Kacheln
AP09	Bewegungslogik	Ermittlung erreichbarer Kacheln
AP10	Ziele und Spezialme-chaniken	Zielkarten, Portale, Tore
AP11	Charaktere	Fähigkeiten und Regelprüfung
AP12	GUI	Spielfeld, Bedienung und Anzeigen
AP13	Server-Client-Kommu-nikation	Server-Client-Kommunikation, Verbindungsaufbau und Synchro-nisation des Spielzustands
AP14	Dokumentation Archi-tektur	Dokumentation der verwendeten Architektur
AP15	Qualitätssicherung	Unit-, Integrations- und Systemtests
AP16	Optimierung	Fehlerbehebung und Benutzerfreundlichkeit
AP17	Projektabschluss	Dokumentation, Benutzeranleitung und finale Abgabe

Quelle: Eigene Darstellung

Für einen Projektzeitraum von sechs Wochen sieht die Planung folgendermaßen aus:

Tabelle 3: Zeitplan

Woche	**Schwerpunkt**	**AP**
1	Projektplanung	AP01, AP02, AP03, AP04
2	Architekturauswahl, Projektsetup & Implementierungsstart	AP05, AP06, AP07, AP08
3	Implementierung	AP09, AP10, AP11
4	Implementierung & Dokumentation	AP12, AP13, AP14
5	Test & Optimierung	AP15, AP16
6	Projektabschluss	AP17

Quelle: Eigene Darstellung

Aus den Arbeitspaketen und der Zeitplanung lassen sich folgende Meilensteine (M) ableiten:

Tabelle 4: Meilensteine

M	AP	Beschreibung
M1: Projektplanung	01–04	Mit diesem Meilenstein sind die grundlegende Projektidee, die Projektplanung, die Anforderungen sowie die fachliche Spezifikation abgeschlossen. Damit liegt eine belastbare Grundlage für die anschließende technische Ausarbeitung und Umsetzung vor.
M2: Architekturauswahl	05	Die grundlegenden technologischen und architektonischen Entscheidungen für die Anwendung sind getroffen und dokumentiert. Dadurch ist festgelegt, auf welcher technischen Basis die weitere Entwicklung erfolgt.
M3: Grundspiel funktionsfähig	06–13	Zu diesem Zeitpunkt sind die wesentlichen Spielfunktionen umgesetzt. Dazu gehören insbesondere Spielfeld, Kachelmechanik, Bewegungslogik, Ziele, Spezialmechaniken, Charaktere und grafische Benutzeroberfläche, sodass eine vollständige Partie grundsätzlich durchgeführt werden kann.
M4: Dokumentation, Tests & Optimierung	14–16	Die verwendete Architektur wurde dokumentiert, die implementierte Anwendung wurde systematisch getestet und erkannte Fehler wurden behoben. Zusätzlich wurden Benutzerfreundlichkeit, Stabilität und das allgemeine Verhalten der Anwendung optimiert.
M5: Projektabschluss	17	Mit diesem Meilenstein sind die Anwendung und die zugehörigen Projektdokumente vollständig fertiggestellt. Die finale Version ist damit für die abschließende Abgabe vorbereitet.

Quelle: Eigene Darstellung

Literaturverzeichnis

Abdellatif, T. M., Capretz, L. F., & Ho, D. (2019). Automatic recall of software lessons learned for software project managers. *Information and Software Technology*, *115*, 44–57. https://doi.org/10.1016/j.infsof.2019.07.006

Di Lucca, G. A., & Fasolino, A. R. (2006). Testing Web-based applications: The state of the art and future trends. *Information and Software Technology*, *48*(12), 1172–1186. https://doi.org/10.1016/j.infsof.2006.06.006

Kuhrmann, M., Diebold, P., Münch, J., Tell, P., Garousi, V., Felderer, M., Trektere, K., McCaffery, F., Linssen, O., Hanser, E., & Prause, C. R. (2017). Hybrid software and system development in practice: waterfall, scrum, and beyond. *Proceedings of the 2017 International Conference on Software and System Process*, *10*, 30–39. https://doi.org/10.1145/3084100.3084104

Lucassen, G., Dalpiaz, F., van der Werf, J. M. E. M., & Brinkkemper, S. (2016). Improving agile requirements: the Quality User Story framework and tool. *Requirements Engineering*, *21*(3), 383–403. https://doi.org/10.1007/s00766-016-0250-x

Mishra, A., & Alzoubi, Y. I. (2023). Structured software development versus agile software development: a comparative analysis. *International Journal of System Assurance Engineering and Management*, *14*(4), 1504–1522. https://doi.org/10.1007/s13198-023-01958-5

Mumtaz, M., Ahmad, N., Usman Ashraf, M., Alghamdi, A. M., Bahaddad, A. A., & Almarhabi, K. A. (2022). Iteration Causes, Impact, and Timing in Software Development Lifecycle: An SLR. *IEEE Access*, *10*, 65355–65375. https://doi.org/10.1109/ACCESS.2022.3182703

Nuseibeh, B., & Easterbrook, S. (2000). Requirements engineering. *Proceedings of the Conference on The Future of Software Engineering*, 35–46. https://doi.org/10.1145/336512.336523

Ravensburger AG. (o. J.). *Das verrückte Labyrinth | Gesellschaftsspiele | Ravensburger*. Abgerufen 26. September 2026, von https://www.ravensburger.de/de-DE/produkte/spiele/gesellschaftsspiele-brettspiele/das-verrueckte-labyrinth-26955